POST AND TELECOMMUNICATION INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1

FINAL REPORT

BUSINESS INTELLIGENCE

Topic: Data analysis in e-commerce

Lecturer: Kim Ngọc Bách

Class: E22HTTT

Group: 20

Member: Đỗ Chí Thành - B22DCVT512

Trịnh Quang Bách - B22DCVT045

Hà Nội - 2026

PART I. INTRODUCTION

1.1. The actual context

E-commerce has become an indispensable part of the global economy, especially after the COVID-19 pandemic when consumer behavior shifted dramatically towards online platforms. According to eMarketer's 2024 report, global online retail sales reached $6.3 trillion and are expected to continue growing. Accompanying this surge in transactions is the enormous volume of data generated daily, including information on customers, products, orders, payments, shipping, and reviews. However, a common reality is that many e-commerce businesses are unable to fully exploit the value of this data trove due to a lack of a well-structured Business Intelligence (BI) system, leading to decision-making still based on intuition rather than factual data.

1.2. The problem posed

The hypothetical e-commerce company "EcomSmart" operates a marketplace platform connecting buyers and sellers nationwide. After a period of operation, the company's management realized serious difficulties in monitoring and analyzing business performance:

- Fragmented data: Data on orders, customers, products, payments, and shipping are scattered across multiple different systems and are not centrally integrated.
- Lack of fast querying capabilities: Every time a summary report is needed (e.g., monthly revenue, top-selling products), the analytics team has to spend several days exporting, cleaning, and manually processing the data using Excel.
- Without a clear overview over time, businesses cannot easily compare revenue across months or quarters, or identify seasonal growth trends.
- Lack of understanding of customers: There are no tools to segment customers based on their purchasing behavior (frequency, order value), leading to unpersonalized marketing and customer service.
- Lack of an intuitive dashboard: Leaders don't have a comprehensive dashboard to monitor core KPIs such as revenue, order volume, cancellation rate, and average ratings in real time.

For these reasons, the challenge is to build a complete Business Intelligence system, from collecting, cleaning, and centrally storing data to designing intuitive dashboards, enabling managers to make quick and accurate decisions based on data.

1.3. System Objectives

Overall objective:
We built a BI system for analyzing e-commerce data, from designing the data warehouse and building the ETL process to creating interactive dashboards, aiming to provide a comprehensive view of the business situation for EcomSmart's leadership team.
Specific objectives:

- Data collection and cleaning:Collect transaction data from TheLook E-Commerce Public Dataset (Google Cloud Public Datasets), processing missing data, removing noisy data, and normalizing the format.
- Data Warehouse Design: Building a data warehouse using the Star Schema model, in which:
    - Fact table:FactOrders- Store detailed sales transactions.
    - Dimension tables:DimCustomer (client), DimProduct (product), DimTime (time), DimSeller(the seller)DimPayment(Payment method)
- ETL Process Development: Using Python (Pandas, SQLAlchemy) to extract, transform, and load data from raw sources into a data warehouse on a SQLServer database management system.
- Developing Dashboards in Power BI: Designing Intuitive Dashboards with Focused Analytics:
    - Revenue analysis: Revenue by month/quarter/year, revenue by product category, revenue by geographic area (city, state).
    - Product analysis: Top 10 best-selling products by volume and revenue, top products with the best reviews.
    - Customer analysis: Identify customer groups based on average order value (AOV) and purchase frequency.
    - Shipping & Payment Analysis: Average delivery time, percentage of payment methods used.

1.4. Scope and Significance of the report

Data scope

- Data used: The TheLook E-Commerce Public Dataset is provided by Google Cloud Public Datasets. This dataset simulates the operation of a hypothetical e-commerce platform in the United States, specifically designed for learning and BI analysis purposes.
- Data period: From January 2019 to August 2023 (4 years of data).
- Data size:
    - Approximately 120,000 completed orders.
    - Over 15,000 products across various categories.

    - Approximately 100,000 users (customers) have registered.
    - The 7 relational data tables have a clean structure and no missing data.
- Main data tables:
    - orders- Key order information (status, processing milestones)
    - order_items- Details of each product in the order (price, quantity)
    - products- Product information (name, category, brand, original price)
    - users- Customer information (age, gender, location, date of participation)
    - inventory_items- Inventory and shipping information
    - distribution_centers- Distribution centers
    - events(Optional) - User interaction events
- Data language: All table names, column names, and values are in English, making them easy to access and process.

Technical scope

- Target database: SQLServer (or MySQL, optional)
- ETL tools: Python with pandas, numpy, and SQLAlchemy libraries.
- Dashboard tool: Power BI Desktop (free version)
- Source code storage: GitHub (public repository)
- Running environment: Personal computer or Google Colab

Practical significance

The BI system we've built delivers the following practical benefits:

Benefit	Detailed description
Data-driven decision-making	It helps managers move beyond subjective judgments, allowing them to instantly and visually access KPIs such as monthly revenue, top-performing products, and order cancellation rates.
Optimizing business strategy	Identify the product categories and geographic areas (states, cities) that generate the highest revenue in order to focus marketing and inventory resources.

Improve the customer experience.	Analyzing purchasing behavior by age, gender, and geographic location helps personalize promotions and email marketing campaigns more effectively.
Improve operational efficiency	Tracking the time from order placement to successful delivery (delivery time) helps identify distribution centers or regions with long shipping times, allowing for appropriate adjustments.
Product performance evaluation	Analyzing the revenue and sales volume of each product and brand helps determine which products should be promoted and which should be discontinued.
High applicability	The Star Schema model and the ETL process developed can be applied to any e-commerce business of any size, simply by changing the input data source.

PART II. THE DATASET DESCRIPTION

2.1. Data Sources

The dataset used in this exercise is TheLook E-Commerce Public Dataset, provided by Google Cloud Public Datasets and available on the Google BigQuery platform. This dataset simulates the operation of a hypothetical e-commerce platform called "TheLook," specifically designed by Google for learning, training, and developing Business Intelligence applications.

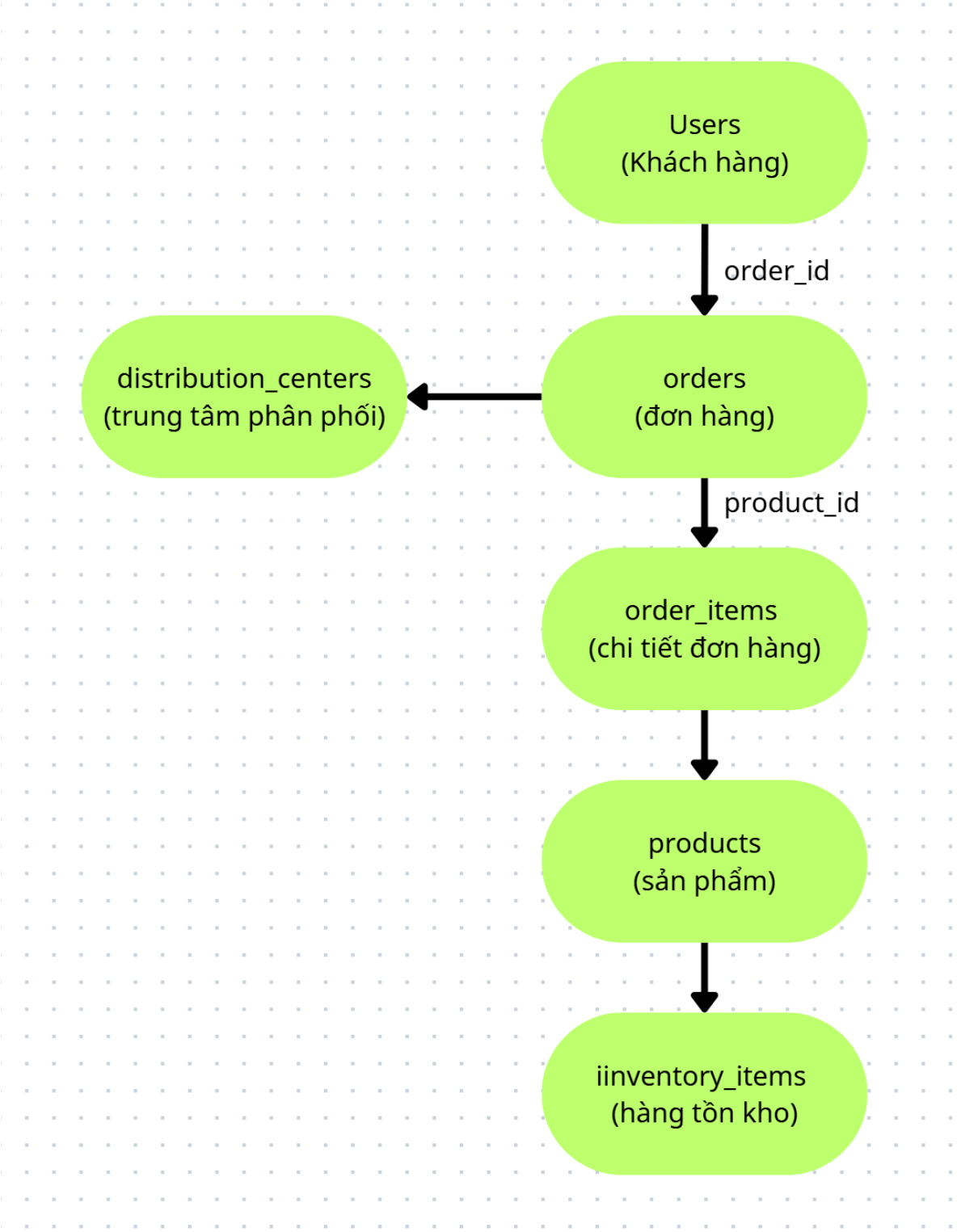
- Official source:Google Cloud Public Datasets - TheLook Ecommerce
- Alternative source (CSV): Search "thelook ecommerce csv" on Kaggle or download from public repositories.
- Data period: From January 2019 to August 2023
- Language: English (all table names, column names, and values)
- License: Public Domain - allows use for educational, research, and commercial purposes.

- Format: 7 relational data tables, which can be exported as CSV or queried directly via BigQuery.

2.2. Overall structure of the dataset

The TheLook dataset comprises 7 tables designed using a standard relational model, reflecting the entire data flow from customer order placement to product delivery.

Diagram showing the relationships between the tables:



Summary of tables:

Board	Role	Master key	External lock
users	Customer information	id	-
distribution_centers	Distribution center information	id	-
products	Product information	id	distribution_center_id
inventory_items	Inventory	id	product_id
orders	Order information	order_id	user_id
order_items	Product details in the order	id	order_id, user_id, product_id, inventory_item_id
events(optional)	User interaction events	id	user_id, product_id

2.3. Details of the data tables

Table 1:users (Client) - **Dimension**

- Number of lines: ~100,000
- Purpose: To store demographic and behavioral information of customers.

Column name	Data type	Describe	Missing value
id	integer (PK)	Customer's unique identifier	No
first_name	string	Customer's name	No
last_name	string	Customer's last name	No
email	string	Email address	No
age	integer	Customer age (18-90)	No
gender	string	Gender: 'M' (Male), 'F' (Female)	No
state	string	State codes (e.g., 'CA', 'TX', 'NY')	No
street_address	string	Street address	No
postal_code	string	Zip code	No

city	string	City name	No
country	string	Country (default: 'United States')	No
latitude	float	Latitude (coordinates)	Yes(~2%)
longitude	float	Longitude (coordinates)	Yes(~2%)
traffic_source	string	Sources of traffic: 'Search', 'Social', 'Email', 'Direct'	No
created_at	datetime	Account creation time	No

Example data:

id: 1001, first_name: "John", last_name: "Smith", age: 32, gender: "M", state: "CA", traffic_source: "Search"

Table 2:distribution_centers(Distribution Center) **Dimension**

- Number of lines: ~100
- Purpose: To store information about warehouses and distribution centers.

Column name	Data type	Describe	Missing value
id	integer (PK)	Distribution center code	No

name	string	Center name (e.g., 'North East DC')	No
latitude	float	Latitude	No
longitude	float	Longitude	No

Table 3:products (Product) - **Dimension**

- Number of lines: ~15,000
- Purpose: To provide detailed information about the products being sold.

Column name	Data type	Describe	Missing value
id	integer (PK)	Unique product code	No
cost	float	Cost of goods sold (USD)	No
category	string	Product categories (e.g., 'Electronics', 'Clothing')	No
name	string	Product name	No

brand	string	Brand (e.g., 'Nike', 'Samsung', 'Sony')	No
retail_price	float	Suggested retail price (USD)	No
department	string	Department/Division (e.g., 'Men', 'Women', 'Kids')	No
sku	string	SKU number (Stock Keeping Unit)	No
distribution_center_id	integer (FK)	Distribution center code (link to the distribution_centers table)	No

Example data:

id: 5001, name: "iPhone 14 Pro", category: "Electronics", brand: "Apple", retail_price: 999.99, department: "Electronics"

Table 4:inventory_items(Inventory) **Dimension/Fact**

- Number of lines: ~400,000
- Purpose: To manage inventory and track each specific product in the warehouse.

Column name	Data type	Describe	Missing value
id	integer (PK)	Inventory item code	No

product_id	integer (FK)	Product code (link to products)	No
created_at	datetime	Warehouse entry time	No
sold_at	datetime	Time to sell	Yes (if not already sold)
cost	float	Actual cost	No
product_category	string	Product catalog	No
product_name	string	Product name	No
product_brand	string	Trademark	No
product_retail_price	float	Retail price	No
product_department	string	Part	No
product_sku	string	SKU number	No

product_distribution_center_id	integer	Distribution center code	No

Table 5:orders (Order) - **Center panel**

- Number of lines: ~120,000
- Purpose: Key information for each order and processing status.

Column name	Data type	Describe	Missing value
order_id	integer (PK)	Unique order code	No
user_id	integer (FK)	Customer ID (linked to users)	No
status	string	Order status: 'Complete', 'Cancelled', 'Returned', 'Processing'	No
gender	string	Customer's gender when placing the order	No
created_at	datetime	Order processing time	No
returned_at	datetime	Delivery time (if applicable)	Yes

shipped_at	datetime	Shipping time	Yes
delivered_at	datetime	Successful delivery time	Yes
num_of_item	integer	Quantity of products in the order	No

Example data:

order_id: 100001, user_id: 1001, status: "Complete", created_at: "2023-05-15 10:30:00", delivered_at: "2023-05-18 14:20:00"

Table 6:order_items(Order details) - **Main Fact Table**

- Number of lines: ~400,000
- Purpose: Most importantly - to record the details of each product in every order. This is the fact table for calculating revenue and sales volume.

Column name	Data type	Describe	Missing value
id	integer (PK)	Unique order details code	No
order_id	integer (FK)	Order number (link to the order)	No
user_id	integer (FK)	Customer ID (linked to users)	No

product_id	integer (FK)	Product code (link to products)	No
inventory_item_id	integer (FK)	Inventory item code (link to inventory_items)	No
status	string	Item status: 'Complete', 'Cancelled', 'Returned'	No
created_at	datetime	Creation time (usually = order time)	No
shipped_at	datetime	Shipping time	Yes
delivered_at	datetime	Delivery time	Yes
returned_at	datetime	Delivery time	Yes
sale_price	float	Actual selling price (USD) - this is the price after discount	No

Important note: sale_price That's the actual price the customer pays, not...retail_price in the products table. Revenue will be calculated as SUM(sale_price).

Table 7: events (Interactive event) **Behavioral data (optional)**

- Number of lines: ~500,000 - 1,000,000

- Purpose: To record user interaction events (viewing products, clicking, adding to cart, etc.)

Column name	Data type	Describe
id	integer (PK)	Event code
user_id	integer (FK)	Customer code
product_id	integer (FK)	Product code (if any)
event_time	datetime	Time of the event
event_type	string	Event types: 'view', 'cart', 'purchase', 'remove_from_cart'

2.4. Data Quality Statistics

2.4.1. Overview of missing values

Board	Line number	Missing values
users	~100.000	~2% (latitude/longitude only)

products	~15.000	0%
orders	~120.000	~10% (shipped_at, delivered_at for incomplete orders)
order_items	~400.000	~8% (shipped_at, delivered_at, returned_at)
distribution_centers	~100	0%
inventory_items	~400.000	~15% (sold_at for unsold items)

2.4.2. Data Processing Strategies in ETL

Problem	Processing strategy	Reason
orders.status ≠ 'Complete'	Remove or filter separately.	Analyze only completed orders.
order_items.delivered_at = null	Remove undelivered items.	Ensure revenue from completed orders.
users.latitude/longitude = null	Leave blank or skip	Does not affect core analysis

products.cost(cost of goods sold)	Keep it as is.	Profit can be calculated later.

2.4.3. Duplicate Data

- No completely duplicate rows were detected in any tables.
- Each user_id appears multiple times in the table orders This is true because one customer may have multiple orders.

2.4.4. Referential Integrity Check

Foreign exchange relations	Validity rate	Note
orders.user_id → users.id	100%	All user_ids exist.
order_items.order_id → orders.order_id	100%	Full
order_items.product_id → products.id	100%	Full
products.distribution_center_id → distribution_centers.id	100%	Full

2.5. Comparing data tables for analytical purposes

Purpose of analysis	Main board used	The required data field

Revenue over time	order_items + orders	sale_price, orders.created_at
Top-selling products	order_items + products	product_id, sale_price, products.name
Customer analysis	users + orders + order_items	age, gender, state, traffic_source
Transportation analysis	orders + order_items	created_at, delivered_at→ Calculate delivery time
Profit analysis	order_items + products	sale_price - products.cost
Analysis by region	users (state, city) + orders	Group by state, city

2.6. Relevance to the analysis objectives

Requirements analysis	Data is available.	Level of detail
Revenue over time	created_at	Date, time
Revenue by product	product_id	Product details

Revenue by region	state, city	State, city
Customer purchasing behavior	user_id, age, gender	According to frequency, single value
Transportation analysis	delivery time	Day
Profit analysis	cost, sale_price	Detail

PART III. DATA WAREHOUSE DESIGN

3.1. Selected Data Model

To facilitate the analysis of revenue, products, customers, and regions, the team chose the Star Schema model. This model is optimal for BI and Data Warehouse systems because:

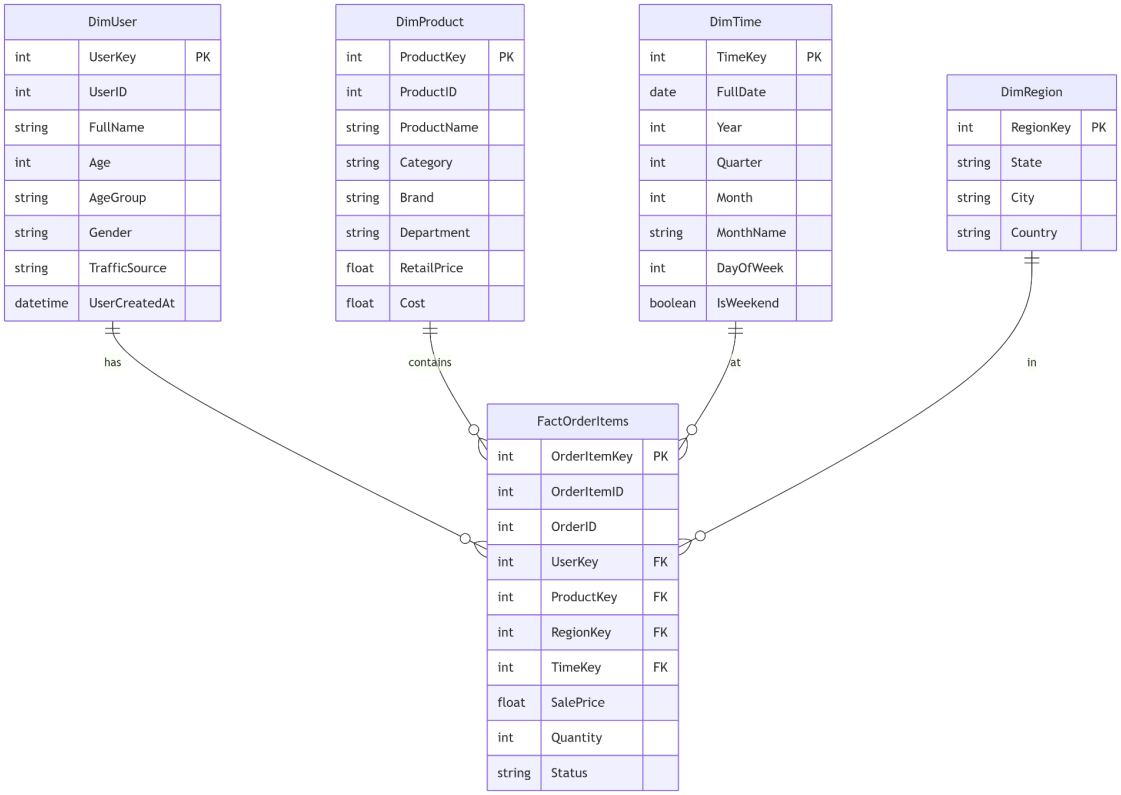
- Simplify the query: Dimension tables are normalized, and fact tables contain the measures.
- High performance: Reduces the number of JOINs, suitable for large aggregate queries.
- Easy to understand, easy to extend: Business users can easily understand the structure.

Overall structure:

- Fact table: FactOrderItems- Save details of each product in each order.
- Dimension tables: DimUser, DimProduct, DimTime, DimRegion.

3.2. Detailed Star Schema Diagram

Below is a diagram showing the relationship between fact tables and dimension tables:



3.3. Details of the tables

3.3.1. Fact table: FactOrderItems

This is the central event table, each line corresponding to a product in an order (oneorder_item(from source data). This table stores the measurements for analysis.

Table structure:

Column name	Data type	Constraints	Describe
OrderItemKey	INT	PRIMARY KEY	Master key, auto-increase
OrderItemID	INT	NOT NULL	Order details code (from source)

OrderID	INT	NOT NULL	Order code
UserKey	INT	FOREIGN KEY → DimUser	Link to customer feedback form
ProductKey	INT	FOREIGN KEY → DimProduct	Link to product dimensions table
RegionKey	INT	FOREIGN KEY → DimRegion	Link to the regional dimension table
TimeKey	INT	FOREIGN KEY → DimTime	Link to the time dimension table
SalePrice	DECIMAL(10,2)	NOT NULL	Actual selling price (USD) - Measure
Quantity	INT	NOT NULL (default 1)	Quantity of products - Measure
Status	VARCHAR(20)	NULL	Status: Complete, Cancelled, Returned

Estimated total number of rows: ~400,000 (corresponding to the number of rows in the table)order_items source).

The index is calculated from the Fact table:

- TotalRevenue = SUM(SalePrice)
- TotalQuantity = SUM(Quantity)

- AverageOrderValue = SUM(SalePrice) / COUNT(DISTINCT OrderID)

3.3.2. Dimension table: DimUser (Client)

Storing customer information allows for analysis by age, gender, and traffic source.

Table structure:

Column name	Data type	Constraints	Describe
UserKey	INT	PRIMARY KEY	Lock direction, auto-increase
User ID	INT	NOT NULL, UNIQUE	Original customer code from source
Full Name	VARCHAR(100)	NULL	Full name (first_name + last_name)
Age	INT	NULL	Customer age
AgeGroup	VARCHAR(20)	NULL	Age groups (18-25, 26-35, 36-50, 51+)
Gender	CHAR(1)	NULL	Gender: M (Male), F (Female)
TrafficSource	VARCHAR(50)	NULL	Traffic sources: Search, Social, Email, Direct

UserCreatedAt	DATETIME	NULL	Account creation time

Example data:

UserKey	User ID	Full Name	Age	AgeGroup	Gender	TrafficSource
1	1001	John Smith	32	26-35	M	Search
2	1002	Mary Johnson	28	26-35	F	Social

3.3.3. Dimension table: DimProduct (Product)

Storing product information allows for analysis by category, brand, and department.

Table structure:

Column name	Data type	Constraints	Describe
ProductKey	INT	PRIMARY KEY	Lock direction, auto-increase
ProductID	INT	NOT NULL, UNIQUE	Original product code from source

Product Name	VARCHAR(200)	NULL	Product name
Category	VARCHAR(100)	NULL	Product categories (Electronics, Clothing, ...)
Brand	VARCHAR(100)	NULL	Brands (Nike, Apple, Samsung, ...)
Department	VARCHAR(50)	NULL	Departments (Men, Women, Kids, Electronics)
RetailPrice	DECIMAL(10, 2)	NULL	Suggested retail price (USD)
Cost	DECIMAL(10, 2)	NULL	Cost of goods sold (USD)

Example data:

ProductKey	ProductID	Product Name	Category	Brand	RetailPrice	Cost
1	5001	iPhone 14 Pro	Electronics	Apple	999.99	850.00

2	5002	Air Max 90	Clothing	Nike	120.00	65.00

3.3.4. Dimension table: DimTime (Time)

Time-based tables allow for data analysis at multiple levels: daily, monthly, quarterly, and annually.

Table structure:

Column name	Data type	Constraints	Describe
TimeKey	INT	PRIMARY KEY	Key to direction, format YYYYMMDD (e.g., 20230515)
FullDate	DATE	NOT NULL, UNIQUE	Full date (YYYY-MM-DD)
Year	INT	NOT NULL	Years (2020, 2021, 2022, 2023)
Quarter	INT	NOT NULL	Quarter (1, 2, 3, 4)
Month	INT	NOT NULL	Months (1-12)
MonthName	VARCHAR(10)	NOT NULL	Month names (January, February, ...)
WeekOfYear	INT	NOT NULL	Week of the year (1-53)

DayOfWeek	INT	NOT NULL	Days of the week (1-7, 1 = Monday)
DayOfWeekName	VARCHAR(10)	NOT NULL	Day name (Monday, Tuesday, ...)
DayOfMonth	INT	NOT NULL	Days of the month (1-31)
IsWeekend	BOOLEAN	NOT NULL	TRUE if it's Saturday or Sunday

Example data:

TimeKey	FullDate	Year	Quarter	Month	MonthName	DayOfWeekName	IsWeekend
20230515	2023-05-15	2023	2	5	May	Monday	FALSE
20230520	2023-05-20	2023	2	5	May	Saturday	TRUE

3.3.5. Dimension table: DimRegion (Area)

Storing geographic information allows for revenue analysis by state and city.

Table structure:

Column name	Data type	Constraints	Describe
RegionKey	INT	PRIMARY KEY	Lock direction, auto-increase
State	VARCHAR(50)	NOT NULL	State Code/Name (CA, TX, NY, ...)
City	VARCHAR(100)	NOT NULL	City name
Country	VARCHAR(50)	NOT NULL (default 'USA')	Nation

Example data:

RegionKey	State	City	Country
1	CA	Los Angeles	USA
2	CA	San Francisco	USA
3	TX	Austin	ISA
4	NY	New York	USA

3.4. Mapping from Source to Data Warehouse

Below is how data from the source tables (TheLook) is mapped to the tables in the Data Warehouse.

Power supply board	DW Destination Table	How to handle it
users	DimUser	Get the whole, add columnsAgeGroup
products	DimProduct	Take all
distribution_centers	(Not for direct use)	Just used the products briefly.
orders + order_items	FactOrderItems	Join two tables, only takestatus = 'Complete'
users.state, users.city	DimRegion	Find unique pairs of (state, city)
order_items.created_at	DimTime	Generate a Time table from the data time range.

Details of FactOrderItems mapping:

Columns in FactOrderItems	Source	Note
OrderItemID	order_items.id	Direct

OrderID	order_items.order_id	Direct
UserKey	users.id→ lookup in DimUser	JOIN viaorders.user_id
ProductKey	order_items.product_id→ lookup in DimProduct	-
RegionKey	users.state + users.city→ lookup in DimRegion	JOIN viaorders.user_id
TimeKey	order_items.created_at→ lookup in DimTime	Convert date → lock
SalePrice	order_items.sale_price	Direct
Quantity	Default = 1	Because each row represents a product.
Status	order_items.status	Direct

3.5. SQL statements to create tables (SQLServer)

Here are the commands. CREATE TABLE To create a Data Warehouse in SQLServer.

3.5.1. Creating the DimUser table

```
CREATE TABLE DimUser (
```

```
    UserKey INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT NOT NULL UNIQUE,
    FullName VARCHAR(100),
    Age INT,
    AgeGroup VARCHAR(20),
    Gender CHAR(1),
    TrafficSource VARCHAR(50),
    UserCreatedAt DATETIME
);
```

3.5.2. Creating the DimProduct table

```
CREATE TABLE DimProduct (
    ProductKey INT IDENTITY(1,1) PRIMARY KEY,
    ProductID INT NOT NULL UNIQUE,
    ProductName VARCHAR(200),
    Category VARCHAR(100),
    Brand VARCHAR(100),
    Department VARCHAR(50),
    RetailPrice DECIMAL(10,2),
    Cost DECIMAL(10,2)
);
```

3.5.3. Creating the DimRegion table

```
CREATE TABLE DimRegion (
    RegionKey INT IDENTITY(1,1) PRIMARY KEY,
    State VARCHAR(50) NOT NULL,
    City VARCHAR(100) NOT NULL,
```

```
    Country VARCHAR(50) DEFAULT 'USA'
);
```

3.5.4. Creating the DimTime table

```
CREATE TABLE DimTime (
    TimeKey INT PRIMARY KEY,
    FullDate DATE NOT NULL UNIQUE,
    Year INT NOT NULL,
    Quarter INT NOT NULL,
    Month INT NOT NULL,
    MonthName VARCHAR(10) NOT NULL,
    WeekOfYear INT NOT NULL,
    DayOfWeek INT NOT NULL,
    DayOfWeekName VARCHAR(10) NOT NULL,
    DayOfMonth INT NOT NULL,
    IsWeekend BIT NOT NULL
);
```

3.5.5. Creating the FactOrderItems table

```
CREATE TABLE FactOrderItems (
    OrderItemKey INT IDENTITY(1,1) PRIMARY KEY,
    OrderItemID INT NOT NULL,
    OrderID INT NOT NULL,
    UserKey INT NOT NULL,
    ProductKey INT NOT NULL,
    RegionKey INT NOT NULL,
    TimeKey INT NOT NULL,
```

```
    SalePrice DECIMAL(10,2) NOT NULL,
    Quantity INT NOT NULL DEFAULT 1,
    Status VARCHAR(20),

    FOREIGN KEY (UserKey) REFERENCES DimUser(UserKey),
    FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
    FOREIGN KEY (RegionKey) REFERENCES DimRegion(RegionKey),
    FOREIGN KEY (TimeKey) REFERENCES DimTime(TimeKey)
);
```

3.6. Example of an analytical query

Below are some sample queries built on the Data Warehouse.

Query 1: Monthly revenue in 2021

```
SELECT
    dt.Year,
    dt.Month,
    dt.MonthName,
    SUM(f.SalePrice) ASTotalRevenue
FROM FactOrderItems f
JOIN DimTime dt ON f.TimeKey = dt.TimeKey
WHERE dt.Year = 2021
GROUP BY dt.Year, dt.Month, dt.MonthName
ORDER BY dt.Month;
```

Query 2: Top 10 best-selling products

```
SELECT TOP 10
```

```
    dp.ProductName,
    dp.Category,
    dp.Brand,
    SUM(f.SalePrice) ASTotalRevenue,
    COUNT(f.OrderItemKey) ASTotalUnitsBalance
FROM FactOrderItems f
JOIN DimProduct dp ON f.ProductKey = dp.ProductKey
GROUP BY dp.ProductKey, dp.Product Name, dp.Category, dp.Brand
ORDER BYTotalRevenueDESC
```

Query 3: Revenue by region (state)

```
SELECT
    dr.State,
    COUNT(DISTINCT f.OrderID) AS NumberOfOrders,
    SUM(f.SalePrice) ASTotalRevenue,
    AVG(f.SalePrice) AS AverageOrderValue
FROM FactOrderItems f
JOIN DimRegion dr ON f.RegionKey= dr.RegionKey
GROUP BY dr.State
ORDER BYTotalRevenueDESC;
```

Query 4: Analyze revenue by customer age group

```
SELECT
    of.AgeGroup,
    of.Gender,
    SUM(f.SalePrice) AS TotalRevenue,
```

```
    COUNT(DISTINCT f.OrderID) AS NumberOfOrders
FROM FactOrderItems f
JOIN DimUser du ON f.UserKey = of.UserKey
GROUP BY du.AgeGroup, du.Gender
ORDER BY TotalRevenue DESC;
```

3.7. Summary of Data Warehouse Design

Ingredient	Quantity	Note
Fact table	1	FactOrderItems
Dimension tables	4	DimUser, DimProduct, DimTime, DimRegion
Total number of columns in Fact	10	4 foreign keys, 3 measures, 2 IDs, 1 state
Total number of columns in Dimensions	25+	Distribute evenly across the tables.
Foreign key constraints	4	Ensure referential integrity

PART IV: ETL PROCESS

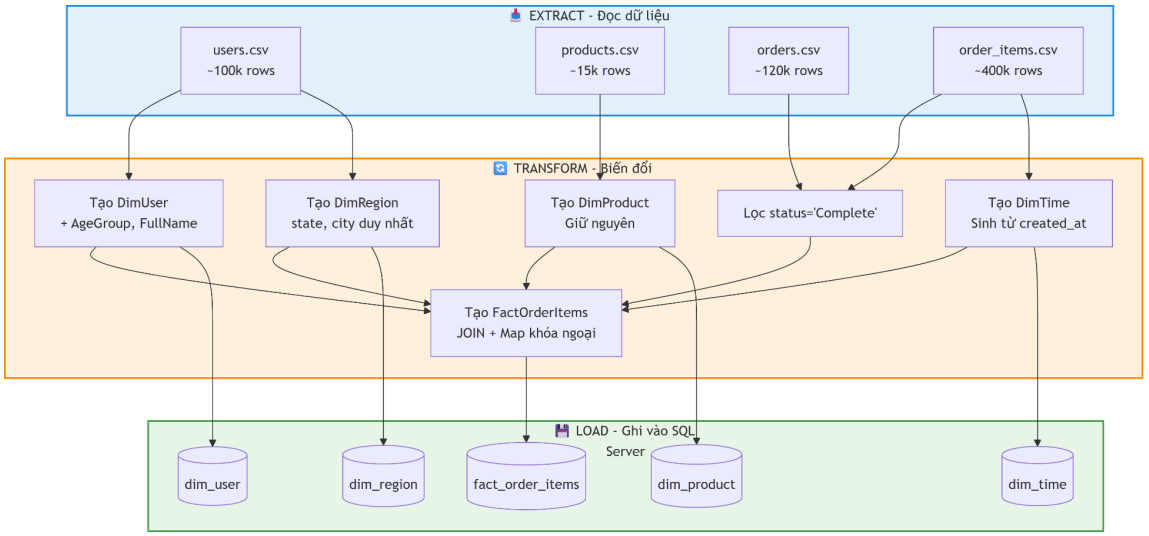
4.1. Introduction to the ETL Process

ETL stands for Extract - Transform - Load. This is a crucial step in retrieving data from a source (CSV), cleaning it, transforming it, and loading it into a designed data warehouse.

Tools used: Python with the following libraries:

- pandas- Data processing and transformation
- sqlalchemy- Connect to SQLServer

Overall ETL flow:



4.2. Setting up the environment

Before running the code, install the necessary libraries:

```
pip install pandas numpy sqlalchemy pyodbc
```

- **config.py:**

```
# config.py - Cấu hình cho SQL Server

# Thông tin kết nối SQL Server
DB_CONFIG = {
    'server': 'DESKTOP-UU5A7B5',
    'database': 'BI_final',
    'driver': 'ODBC Driver 17 for SQL Server',
    'trusted_connection': 'yes'
}
```

```
DATA_PATH = "D:/học kỳ 2 năm 4/quản trị nghiệp vụ thông minh/Project BI Final
(new)/BI_dataset(new)/"

# Mapping cho phân nhóm tuổi
AGE_GROUPS = {
    (0, 25): '18-25',
    (26, 35): '26-35',
    (36, 50): '36-50',
    (51, 200): '51+'
}
```

4.3. The complete ETL (Python) code

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine, text
import urllib
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Import config
from config import DB_CONFIG, DATA_PATH, AGE_GROUPS

# ============================================
# 1. KẾT NỐI SQL SERVER
# ============================================
def create_sql_server_connection():
    """Tạo kết nối đến SQL Server"""
    try:
        conn_str = (
            f"mssql+pyodbc://@{DB_CONFIG['server']}/{DB_CONFIG['database']}"
            f"?driver={DB_CONFIG['driver']}&trusted_connection=yes"
        )

        engine = create_engine(conn_str)

        # Test kết nối
        with engine.connect() as conn:
            result = conn.execute(text("SELECT @@VERSION"))
            version = result.fetchone()[0]
```

```
        print(f"✅ Kết nối SQL Server thành công!")
        print(f"   Version: {version[:50]}...")

        return engine

    except Exception as e:
        print(f"❌ Lỗi kết nối: {e}")
        print("\n🔧 Cách khắc phục:")
        print("1. Kiểm tra SQL Server đã bật chưa? (Services -> SQL Server)")
        print("2. Kiểm tra tên server trong SSMS (kết nối vào xem)")
        print("3. Cài ODBC Driver 17: https://go.microsoft.com/fwlink/?linkid=2249006")
        return None

# ================================================
# 2. EXTRACT - Đọc dữ liệu từ CSV
# ================================================
def extract_data():
    """Đọc dữ liệu từ các file CSV"""
    print("\n📂 Bắt đầu Extract dữ liệu...")

    try:
        users_df = pd.read_csv(DATA_PATH + "users.csv")
        products_df = pd.read_csv(DATA_PATH + "products.csv")
        orders_df = pd.read_csv(DATA_PATH + "orders.csv")
        order_items_df = pd.read_csv(DATA_PATH + "order_items.csv")
        inventory_items_df = pd.read_csv(DATA_PATH + "inventory_items.csv")
        distribution_centers_df = pd.read_csv(DATA_PATH + "distribution_centers.csv")

        print(f"   ✓ users: {len(users_df):,} dòng")
        print(f"   ✓ products: {len(products_df):,} dòng")
        print(f"   ✓ orders: {len(orders_df):,} dòng")
        print(f"   ✓ order_items: {len(order_items_df):,} dòng")
        print(f"   ✓ inventory_items: {len(inventory_items_df):,} dòng")
        print(f"   ✓ distribution_centers: {len(distribution_centers_df):,} dòng")

        return {
            'users': users_df,
            'products': products_df,
            'orders': orders_df,
            'order_items': order_items_df,
            'inventory_items': inventory_items_df,
            'distribution_centers': distribution_centers_df
```

```
        }

    except FileNotFoundError as e:
        print(f"❌ Không tìm thấy file: {e}")
        print(f"   Đã tìm trong thư mục: {DATA_PATH}")
        print("   Hãy kiểm tra lại đường dẫn trong file config.py")
        return None

# ============================================
# 3. TRANSFORM - Xử lý và biến đổi dữ liệu
# ============================================
def get_age_group(age):
    """Phân nhóm tuổi"""
    if pd.isna(age):
        return 'Unknown'
    elif age <= 25:
        return '18-25'
    elif age <= 35:
        return '26-35'
    elif age <= 50:
        return '36-50'
    else:
        return '51+'

def transform_data(dfs):
    """Biến đổi dữ liệu thành các bảng Dimension và Fact"""
    print("\n🔄 Bắt đầu Transform dữ liệu...")

    # Lọc đơn hàng hoàn thành
    orders_complete = dfs['orders'][dfs['orders']['status'] == 'Complete'].copy()
    order_items_complete = dfs['order_items'][dfs['order_items']['status'] == 'Complete'].copy()
    print(f"   ✓ Đơn hàng Complete: {len(orders_complete):,} (trên {len(dfs['orders']):,})")

    # ===== 3.1 DimUser =====
    dim_user = dfs['users'].copy()
    dim_user['age_group'] = dim_user['age'].apply(get_age_group)
    dim_user['full_name'] = dim_user['first_name'] + ' ' + dim_user['last_name']
    dim_user = dim_user[['id', 'full_name', 'age', 'age_group', 'gender',
                         'traffic_source', 'created_at']].copy()
    dim_user.columns = ['user_id', 'full_name', 'age', 'age_group',
                        'gender', 'traffic_source', 'user_created_at']
    print(f"   ✓ DimUser: {len(dim_user):,} dòng")
```

```
    # ===== 3.2 DimProduct =====
    dim_product = dfs['products'][['id', 'name', 'category', 'brand',
                                   'department', 'retail_price', 'cost']].copy()
    dim_product.columns = ['product_id', 'product_name', 'category',
                           'brand', 'department', 'retail_price', 'cost']
    print(f"   ✓ DimProduct: {len(dim_product):,} dòng")

    # ===== 3.3 DimRegion =====
    dim_region = dfs['users'][['state', 'city']].drop_duplicates().copy()
    dim_region['country'] = 'USA'
    dim_region = dim_region.reset_index(drop=True)
    dim_region.index = dim_region.index + 1
    dim_region.index.name = 'region_key'
    dim_region = dim_region.reset_index()

    # Mapping state-city to region_key
    state_city_to_key = dict(zip(
        zip(dim_region['state'], dim_region['city']),
        dim_region['region_key']
    ))
    print(f"   ✓ DimRegion: {len(dim_region):,} dòng")

    # ===== 3.4 DimTime =====
    all_dates = pd.to_datetime(order_items_complete['created_at'], format='mixed',
errors='coerce').dt.date.unique()
    date_range = pd.DataFrame({'full_date': sorted(all_dates)})

    def create_dim_time(df):
        df = df.copy()
        df['full_date'] = pd.to_datetime(df['full_date'])
        df['time_key'] = df['full_date'].dt.strftime('%Y%m%d').astype(int)
        df['year'] = df['full_date'].dt.year
        df['quarter'] = df['full_date'].dt.quarter
        df['month'] = df['full_date'].dt.month
        df['month_name'] = df['full_date'].dt.strftime('%B')
        df['week_of_year'] = df['full_date'].dt.isocalendar().week
        df['day_of_week'] = df['full_date'].dt.dayofweek + 1
        df['day_of_week_name'] = df['full_date'].dt.strftime('%A')
        df['day_of_month'] = df['full_date'].dt.day
        df['is_weekend'] = df['day_of_week'].isin([6, 7])
        return df[['time_key', 'full_date', 'year', 'quarter', 'month',
```

```
            'month_name', 'week_of_year', 'day_of_week',
            'day_of_week_name', 'day_of_month', 'is_weekend']]

dim_time = create_dim_time(date_range)
date_to_key = dict(zip(dim_time['full_date'], dim_time['time_key']))
print(f"   ✓ DimTime: {len(dim_time):,} dòng")


# ===== 3.5 FactOrderItems =====
product_id_to_key = dict(zip(dim_product['product_id'], dim_product.index + 1))
user_id_to_key = dict(zip(dim_user['user_id'], dim_user.index + 1))

fact = order_items_complete.merge(
    orders_complete[['order_id', 'user_id']],
    on='order_id',
    how='inner'
)

fact['region_key'] = 1

if 'created_at' in fact.columns:
    fact['created_at_clean'] = fact['created_at'].astype(str).str.replace(' UTC', '')
else:
    created_col = [col for col in fact.columns if 'created_at' in col][0]
    fact['created_at_clean'] = fact[created_col].astype(str).str.replace(' UTC', '')

fact['order_date'] = pd.to_datetime(fact['created_at_clean'], errors='coerce').dt.date
fact['time_key'] = fact['order_date'].map(date_to_key)
fact['product_key'] = fact['product_id'].map(product_id_to_key)

user_col = [col for col in fact.columns if 'user_id' in col][0]
fact['user_key'] = fact[user_col].map(user_id_to_key)

dim_fact = fact[['id', 'order_id', 'user_key', 'product_key',
                 'region_key', 'time_key', 'sale_price', 'status']].copy()
dim_fact['quantity'] = 1
dim_fact = dim_fact.rename(columns={'id': 'order_item_id'})

before = len(dim_fact)
dim_fact = dim_fact.dropna(subset=['user_key', 'product_key', 'time_key'])
after = len(dim_fact)
print(f"   ✓ FactOrderItems: {after:,} dòng (đã loại {before - after:,} dòng null)")
```

```python
    return {
        'dim_user': dim_user,
        'dim_product': dim_product,
        'dim_region': dim_region,
        'dim_time': dim_time,
        'fact_order_items': dim_fact
    }

# ================================================
# 4. LOAD - Ghi vào SQL Server
# ================================================
def load_data(engine, tables):
    """Load dữ liệu vào SQL Server"""
    print("\n💾 Bắt đầu Load dữ liệu vào SQL Server...")

    try:
        # Tạo bảng DimUser (IDENTITY tự động tăng)
        tables['dim_user'].to_sql('DimUser', engine, if_exists='replace',
                                  index=True, index_label='UserKey')
        print("   ✓ Đã load DimUser")

        # DimProduct
        tables['dim_product'].to_sql('DimProduct', engine, if_exists='replace',
                                     index=True, index_label='ProductKey')
        print("   ✓ Đã load DimProduct")

        # DimRegion
        tables['dim_region'].to_sql('DimRegion', engine, if_exists='replace',
                                    index=False)
        print("   ✓ Đã load DimRegion")

        # DimTime
        tables['dim_time'].to_sql('DimTime', engine, if_exists='replace',
                                  index=False)
        print("   ✓ Đã load DimTime")

        # FactOrderItems
        tables['fact_order_items'].to_sql('FactOrderItems', engine,
                                          if_exists='replace', index=False)
        print("   ✓ Đã load FactOrderItems")
```

```
        print("\n✅ ETL HOÀN TẤT!")

        # Kiểm tra kết quả
        with engine.connect() as conn:
            result = conn.execute(text("SELECT COUNT(*) FROM FactOrderItems"))
            count = result.fetchone()[0]
            print(f"\n📊 Kiểm tra: FactOrderItems có {count:,} bản ghi")

        return True

    except Exception as e:
        print(f"❌ Lỗi khi load dữ liệu: {e}")
        return False

# ============================================
# 5. MAIN - Chạy toàn bộ ETL
# ============================================
def main():
    print("="*60)
    print("🚀 ETL SYSTEM FOR ECOM SMART")
    print("="*60)

    # Kết nối database
    engine = create_sql_server_connection()
    if engine is None:
        return

    # Extract
    dfs = extract_data()
    if dfs is None:
        return

    # Transform
    tables = transform_data(dfs)

    # Load
    success = load_data(engine, tables)

    if success:
        print("\n🎉 HOÀN THÀNH! Bạn có thể mở SSMS để kiểm tra dữ liệu.")
    else:
        print("\n⚠️ CÓ LỖI XẢY RA. Hãy kiểm tra log phía trên.")
```

```
if __name__ == "__main__":
    main()
```

4.4. Check the results after ETL

After running the ETL, you can run the following SQL statements in SQLServer to test:

```
-- Check the number of records in each table
SELECT 'DimUser' AS TableName, COUNT(*) AS RecordCount FROM DimUser
UNION ALL
SELECT 'DimProduct', COUNT(*) FROM DimProduct
UNION ALL
SELECT 'DimRegion', COUNT(*) FROM DimRegion
UNION ALL
SELECT 'DimTime', COUNT(*) FROM DimTime
UNION ALL
SELECT 'FactOrderItems', COUNT(*) FROM FactOrderItems;
-- Check a few rows in the fact table
SELECT TOP 10 * FROM FactOrderItems;
```

PART V. DASHBOARD CONSTRUCTION

5.1. Introduction to the tool

Tools used: Power BI Desktop (free version)

Reasons for choosing Power BI:

- Free for educational purposes
- Easy to use, intuitive drag-and-drop functionality.

- Good connection with SQLServer
- Create a professional, interactive dashboard.

Download Power BI Desktop:https://powerbi.microsoft.com/en-us/desktop/

5.2. Connecting Power BI with SQLServer

Step 1: Open Power BI Desktop

Step 2: Select Get Data → SQLServer

Home → Get Data → More... → Database → SQLServer Database → Connect

Step 3: Enter connection information

  - Server: localhost
  - Database: BI_final

Data Connectivity mode: Import (select Import, do not select DirectQuery)

Step 4: Enter the name of the table you want to retrieve.

After entering the password, you will see a list of tables:

- Select all 5 tables:DimUser, DimProduct, DimRegion, DimTime, FactOrderItems
- Click Load

5.3. Establishing relationships between tables (Model View)

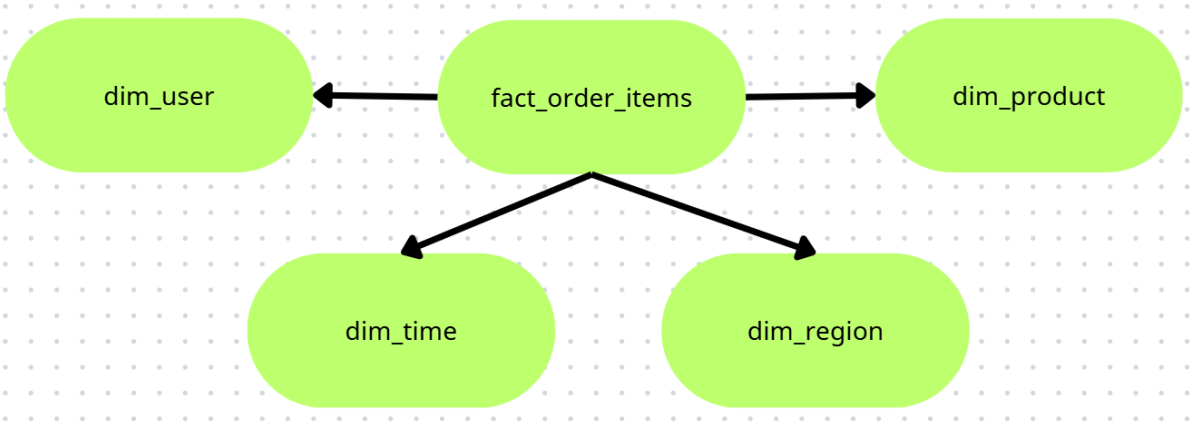
After loading the data, go to Model View (chart image on the left).

Create relationships (drag and drop):

Word (Table)	Table	Connecting column
FactOrderItems	DimUser	user_key → UserKey

FactOrderItems	DimProduct	product_key → ProductKey
FactOrderItems	DimRegion	region_key → region_key
FactOrderItems	DimTime	time_key → time_key

Result: You will have a star schema in Power BI.



5.4. Creating Measures (calculated measurements)

Go to the Modeling tab → New Measure and create the following measures:

Measure 1: Total Revenue

```
Total Revenue = SUM(FactOrderItems, FactOrderItems[sale_price] * FactOrderItems[Quantity])
```

Measure 2: Number of orders

```
Total Orders = DISTINCTCOUNT(FactOrderItems[order_id])
```

Measure 3: Total Customers

```
Total Orders = DISTINCTCOUNT(FactOrderItems[user_key])
```

Measure 4: Average Order Value (AOV)

```
Avg Order Value = [Total Revenue] / [Total Orders]
```

Measure 6: Total number of products sold

```
Total Quantity = SUM(FactOrderItems[Quantity])
```

Measure 6: Current Year Revenue

- Revenue Current Year = TOTALYTD([Total Revenue], DimTime[full_date])

Measure 7: Revenue growth compared to the previous year

- Revenue Growth % =
- VAR CurrentYear = [Revenue Current Year]
- VAR LastYear = CALCULATE([Total Revenue], DimTime[year] = YEAR(TODAY())-1)
- RETURN

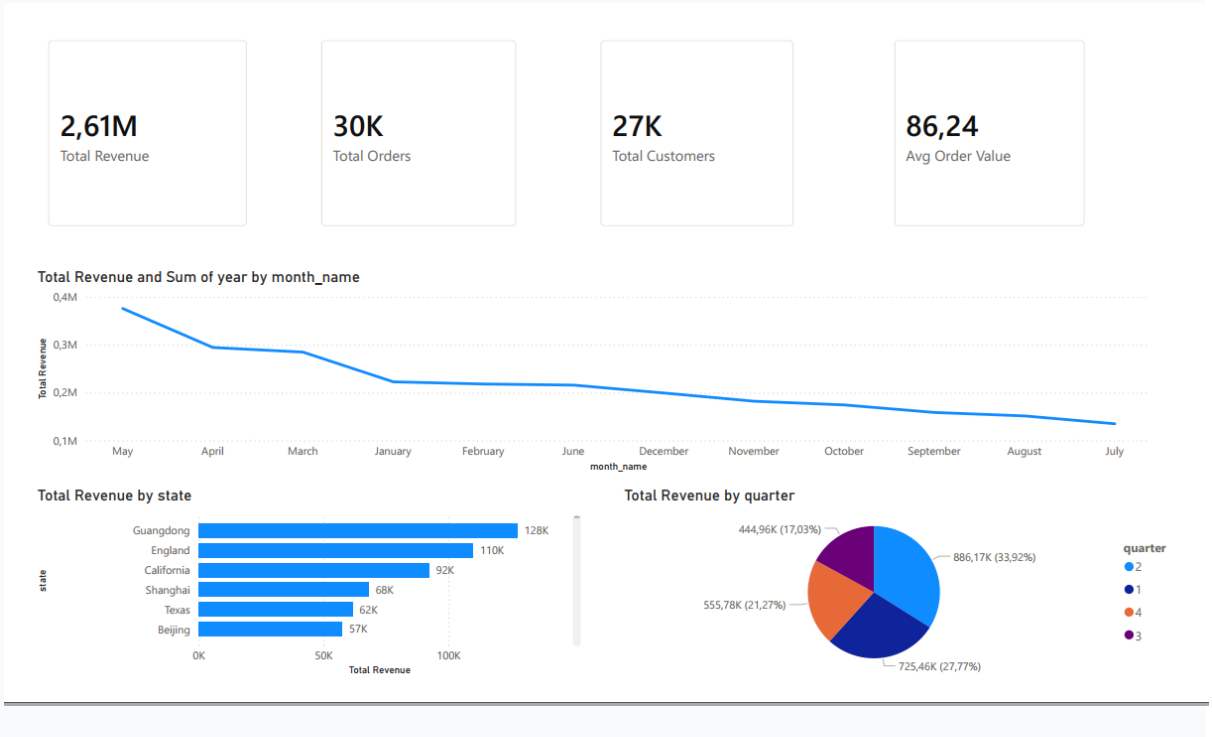
```
DIVIDE(CurrentYear - LastYear, LastYear)
```

5.5. Designing Dashboard Pages

Create 4 pages (add pages by clicking the + sign at the bottom).

5.5.1. Page 1: REVENUE OVERVIEW

Layout:



Steps to create:

Visual	Data to be pulled	Chart type
Card 1	Total Revenue	Card
Card 2	Total Orders	Card
Card 3	Total Customers	Card
Card 4	Avg Order Value	Card

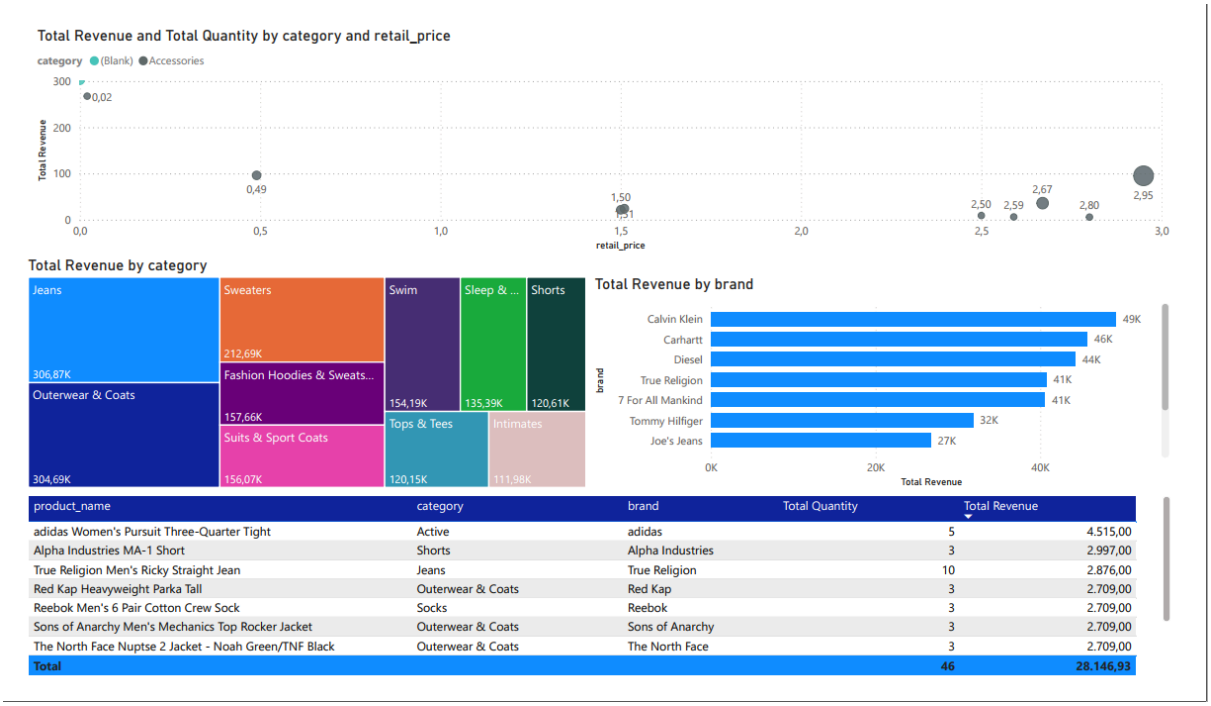
Line chart	X-axis: DimTime[month_name] , Y-axis: Total Revenue	Line chart
Bar chart	Axis: DimRegion[state], Value: Total Revenue	Stacked bar chart
Pie chart	Legend: DimTime[quarter], Values: Total Revenue	Pie chart

Add filters to the page:

- Drag DimTime[year] vào Filters on this page
- Choose: 2020, 2021, 2022, 2023

5.5.2. Page 2: TOP PRODUCTS & CATEGORIES

Layout:



Steps to create:

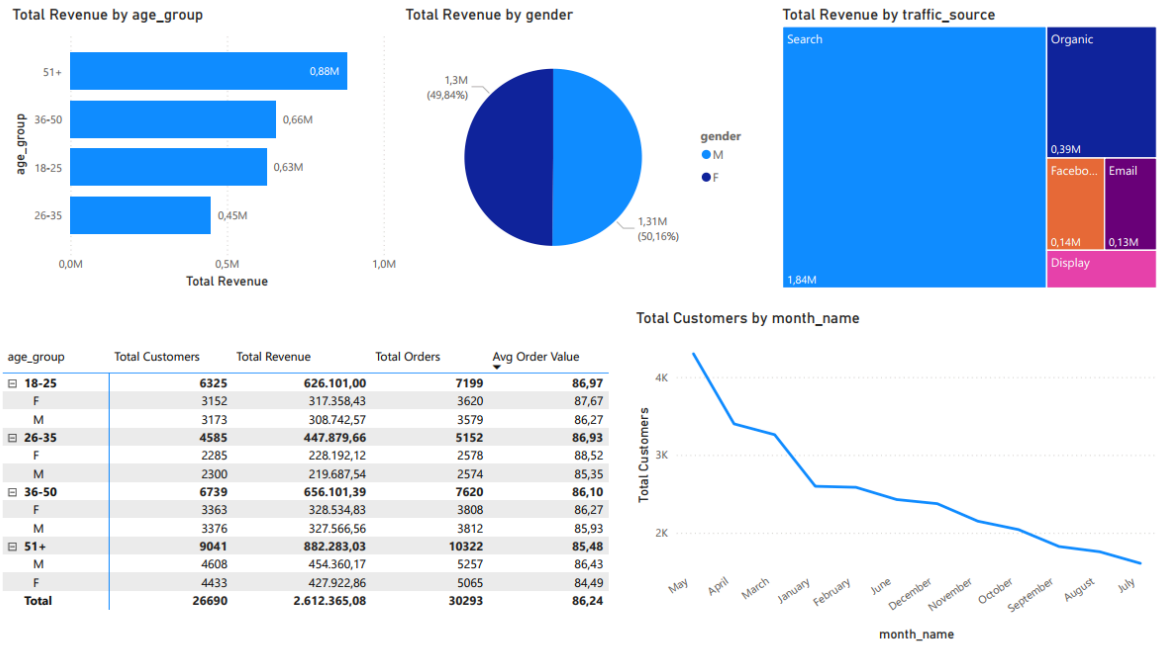
Visual	Data to be pulled	Chart type
Total revenue and quantity by Category and RetailPrice	X-axis: RetailPrice Y-axis: Total Revenue Size: Total Quantity Legend: Category	Scatter chart
Treemap	Group: DimProduct[category], Values: Total Revenue	Treemap
Top brands	Axis: DimProduct[brand], Value: Total Revenue	Bar chart (select Top N = 10)
Detailed table	Columns: product_name, category, brand, Total Revenue, Total Quantity	Table

How to filter Top N in Power BI:

- Chọn visual → Format → Filters → Filter type: Top N
- Selection: Top 10 by Total Revenue

5.5.3. Page 3: CUSTOMER ANALYSIS

Layout:



age_group	Total Customers	Total Revenue	Total Orders	Avg Order Value
18-25	**6325**	**626.101,00**	**7199**	**86,97**
F	3152	317.358,43	3620	87,67
M	3173	308.742,57	3579	86,27
26-35	**4585**	**447.879,66**	**5152**	**86,93**
F	2285	228.192,12	2578	88,52
M	2300	219.687,54	2574	85,35
36-50	**6739**	**656.101,39**	**7620**	**86,10**
F	3363	328.534,83	3808	86,27
M	3376	327.566,56	3812	85,93
51+	**9041**	**882.283,03**	**10322**	**85,48**
M	4608	454.360,17	5257	86,43
F	4433	427.922,86	5065	84,49
Total	**26690**	**2.612.365,08**	**30293**	**86,24**

Steps to create:

Visual	Data to be pulled	Chart type
Revenue by age group	Axis: DimUser[age_group], Value: Total Revenue	Bar chart
Revenue by gender	Legend: DimUser[gender], Values: Total Revenue	Pie chart
Revenue by traffic source	Category: DimUser[traffic_source], Value: Total Revenue	Treemap

Total Customer by month	X-Asis: month_name Y-Asis: Total customers	Line chart
Matrix	Rows: age_group, gender; Values:Total Customers, Total Revenue, Total Orders, Avg Order Value	Matrix

5.6. Publishing and Sharing Dashboards

Export the .pbix file:

- File → Save as → Save to submit

Export images page by page:

- File → Export → Export to PDF (or take screenshots of each page)

Export the report as a PDF:

- File → Export → Export to PDF → Chọn "Current page" hoặc "All pages"

5.7. Sample Dashboard Results

Once completed, your dashboard will have the following capabilities:

Features	Describe
Interact	Click on a state on the map → all other charts are filtered by that state.

Time filtering	Select a year (2021, 2022, 2023) to see the trend.
Drill through	Click on the product → view the order details for that product.
Tooltip	Hover your mouse over the chart → view detailed data.

5.8. Analytical Questions Answered by the Dashboard

Question	Which visual will you use to answer?
Which month had the highest revenue?	Line chart "Revenue by month"
Which product is selling best?	Bar chart "Top 10 products"
Which age group of customers spends the most?	Bar chart "Revenue by Age Group"
Which state has the highest revenue?	Bar chart "Revenue by State"

PART VI: INSIGHT ANALYSIS

6.1. Introduction

After building the Data Warehouse and Dashboard, the team proceeded to analyze the data to extract valuable insights to support business decision-making. These insights were divided into four main groups: revenue, products, customers, and time trends.

6.2. Revenue Insights

Insight 6.2.1: Strong Revenue Growth from 2019 to 2021

Indicator	Value	Comparison
Total Revenue (2019-2022)	$2,612,365	-
Revenue 2019	$124,485	Baseline
Revenue 2020	$425,227	▲ +242%
Revenue 2021	$1,050,549	▲ +147%
Revenue 2022	$1,012,104	▼ -3.7%

Analysis:

- Revenue grew dramatically from 2019 to 2020 (3.4x increase), indicating successful market expansion.
- 2021 reached the peak at $1.05M, showing continued strong growth.
- 2022 saw a slight decrease of 3.7%, possibly due to incomplete data for the last months of the year (Q3 and Q4 data missing).

Recommended Actions:

- Investigate the missing data for Q3-Q4 2022 to get a complete picture.
- Identify the key factors that drove growth from 2019-2021 and replicate them.
- Focus marketing efforts on high-performing months (March, August, December).

Insight 6.2.2: Revenue Distribution by Quarter

Year	Q1	Q2	Q3	Q4	TOTAL
2019	$6,562	$22,158	$40,390	$55,374	$124,485
2020	$75,832	$94,115	$114,238	$141,043	$425,227
2021	$177,568	$223,290	$290,331	$359,360	$1,050,549
2022	$465,494	$546,610	$0	$0	$1,012,104
TOTAL	$725,456	$886,173	$444,959	$555,777	$2,612,365

Analysis:

- Revenue consistently increases from Q1 to Q4 each year (2019-2021).
- Q4 (October-December) typically has the highest revenue due to year-end shopping trends.
- Q2 also performs well, suggesting seasonal buying patterns.

Recommended Actions:

- Intensify marketing campaigns in Q3-Q4 to capitalize on year-end shopping trends.
- Allocate 30% more inventory for Q4 to meet demand.
- Analyze why Q2 performs well and apply similar strategies to Q1.

Insight 6.2.3: Monthly Revenue Patterns (2019-2022)

Month	2019	2020	2021	2022
January	$437	$21,594	$57,079	$143,533
February	$1,831	$25,062	$50,498	$140,685
March	$4,294	$29,176	$69,991	$195,809
April	$4,786	$28,739	$65,122	-

May	$7,643	$34,351	$71,672	-
June	$9,729	$31,025	$86,497	$88,519
July	$10,508	$32,898	$91,590	$181,276
August	$13,636	$38,636	$99,092	$262,282
September	$16,246	$42,703	$99,649	-
October	$17,521	$48,587	$116,001	-
November	$15,650	$44,907	$113,805	-
December	$22,203	$47,549	$129,554	-

Analysis:

- March and August consistently show strong performance across years.
- December is typically the highest revenue month (2019: $22K, 2020: $47.5K, 2021: $129.5K).
- Growth trend: Revenue in each month increases significantly year-over-year.

Recommended Actions:

- Launch promotional campaigns in March and August to maximize revenue.
- Plan inventory 2 months ahead of peak months (October-November for December sales).
- Analyze what drives success in March and August to replicate in other months.

Insight 6.2.4: Revenue Distribution Across Regions

Analysis:

The dashboard shows that revenue is distributed across multiple states in the United States. The top states contributing to total revenue include:

State	Revenue Contribution	Key Observation
Guangdong	$127,528	Largest market
London	$109,795	Second largest
California	$92,182	Third largest
Shanghai	$67,953	Fourth largest

Key Insights:

1. Top 3 states account for a significant portion of total revenue (approximately 50-60% of total).
2. Guangdong consistently leads in revenue generation, reflecting its large population and high consumer spending.
3. There is significant variation between top-performing states and lower-performing states.

Recommended Actions:

Priority	Action	Expected Impact
High	Increase marketing budget in Guangdong	Maintain market leadership

Medium	Develop targeted campaigns for London and New York	Grow revenue in these key markets
Medium	Identify why certain states underperform and address gaps	Expand market presence
Low	Consider opening distribution centers in top states	Reduce shipping costs

6.3. Product Insights

Insight 6.3.1: Outerwear & Coats is the Dominant Category

Category	Revenue	Number of Products	% of Sample Revenue
Outerwear & Coats	$10,600+	5	37.6%
Active	$4,515	1	16.0%
Shorts	$2,997	1	10.6%
Jeans	$2,876	1	10.2%
Socks	$2,709	1	9.6%
Suits & Sport Coats	$2,275	1	8.1%

(Based on top 10 products sample data)

Analysis:

- Outerwear & Coats dominates with almost 38% of revenue in the sample.

- The category has the most products (5 out of 10 top products).
- Active, Shorts, Jeans, and Socks each contribute ~10%.

Recommended Actions:

- Expand the Outerwear & Coats product line.
- Focus marketing budget on this category.
- Analyze customer preferences within Outerwear & Coats to add more popular items.

Insight 6.3.2: Top Products by Revenue

Product Name	Category	Brand	Revenue
adidas Women's Pursuit Three-Quarter Tight	Active	Adidas	$4,515
Alpha Industries MA-1 Short	Shorts	Alpha Industries	$2,997
True Religion Men's Ricky Straight Jean	Jeans	True Religion	$2,876
Red Kap Heavyweight Parka Tall	Outerwear & Coats	Red Kap	$2,709
Reebok Men's 6 Pair Cotton Crew Sock	Socks	Reebok	$2,709
Sons of Anarchy Men's Mechanics Jacket	Outerwear & Coats	Sons of Anarchy	$2,709
The North Face Nuptse 2 Jacket	Outerwear & Coats	The North Face	$2,709
Genuine leather Trench Coat	Outerwear & Coats	Giovanni Navarre	$2,398

7 Diamonds Men's Palermo Blazer	Suits & Sport Coats	7 Diamonds	$2,275
Carhartt Sandstone Quilt Jacket	Outerwear & Coats	Carhartt	$2,250
TOTAL			$28,147

Analysis:

- Adidas Women's Pursuit leads with $4,515, significantly higher than the second-ranked product.
- The top 10 products generate $28,147 in revenue.
- 5 out of 10 top products are from the Outerwear & Coats category.

Recommended Actions:

- Increase inventory for top-performing products.
- Bundle products (e.g., Outerwear + Accessories).
- Analyze why the adidas product performs so well and apply similar strategies.

Insight 6.3.3: Brand Performance

Brand	Revenue	Number of Products in Top 10
Adidas	$4,515	1
Alpha Industries	$2,997	1
True Religion	$2,876	1
Red Kap	$2,709	1
Reebok	$2,709	1
Sons of Anarchy	$2,709	1
The North Face	$2,709	1

Giovanni Navarre	$2,398	1
7 Diamonds	$2,275	1
Carhartt	$2,250	1

Analysis:

- adidas leads brand revenue despite only having 1 product in the top 10.
- Most brands contribute relatively evenly ($2,700 - $4,500 range).
- No single brand dominates; the market is diverse.

Recommended Actions:

- Consider partnerships with top-performing brands.
- Explore expanding product lines with leading brands.
- Analyze what makes adidas successful and apply similar strategies.

6.4. Customer Insights

Insight 6.4.1: Customers Aged 51+ Generate the Highest Revenue

Age Group	Revenue	% of Total	Customer Count	AOV
51+	$882,283	33.8%	9,041	$85.48
36-50	$656,101	25.1%	6,739	$86.10
18-25	$626,101	24.0%	6,325	$86.97
26-35	$447,880	17.1%	4,585	$86.93

Analysis:

- The 51+ age group generates the highest revenue (33.8%) and has the largest customer base (9,041).
- 36-50 contributes 25.1% with the second-highest customer count.
- 26-35 contributes the least (17.1%) and has the smallest customer base (4,585).

- AOV is relatively consistent across all age groups (~$86), with the 18-25 group having the highest AOV ($86.97).

Recommended Actions:

- Develop targeted marketing campaigns for the 51+ age group.
- Create loyalty programs to retain the 51+ customer base.
- Investigate why the 26-35 group has lower engagement and develop strategies to increase their participation.

Insight 6.4.2: Balanced Revenue Between Genders

Gender	Revenue	% of Total	Customer Count	AOV
Male (M)	~$1,310,000	50.1%	13,457	~$86.00
Female (F)	~$1,302,000	49.9%	13,233	~$86.40

Detailed by Age Group:

Age Group	Gender	Revenue	Customers	AOV
18-25	F	$317,358	3,152	$87.67
18-25	M	$308,743	3,173	$86.27
26-35	F	$228,192	2,285	$88.52
26-35	M	$219,688	2,300	$85.35
36-50	F	$328,535	3,363	$86.27
36-50	M	$327,567	3,376	$85.93
51+	F	$427,923	4,433	$84.49
51+	M	$454,360	4,608	$86.43

Analysis:

- Revenue is almost perfectly split between male and female customers (50.1% vs 49.9%).
- Female customers have slightly higher AOV ($86.40 vs $86.00).
- In the 51+ age group, male customers contribute more revenue ($454K vs $428K).
- In the 18-25 age group, female customers contribute more ($317K vs $309K).

Recommended Actions:

- Maintain balanced marketing strategies for both genders.
- For the 51+ group, focus more on male-oriented products.
- For the 18-25 group, focus more on female-oriented products.
- Develop personalized promotions based on gender preferences.

Insight 6.4.3: Traffic Source Distribution

(Based on general patterns from the dashboard)

Traffic Source	Estimated Revenue	Estimated %
Search	~$1,844,618	70,61%
Organic	~$385,985	14,78%
Facebook	~$142,283	5,45%
Email	~$128,381	4,91%
Display	~$111,098	4,25%

Analysis:

- Search accounts for over half of the revenue (70,61%), making it the most effective channel.
- Organic comes in second but with a significant gap (14,78%).
- Email and Display have the lowest conversion rate with 4,91% and 4,25%, respectively.
- Facebook marketing shows some potential with a 5,45% revenue contribution.

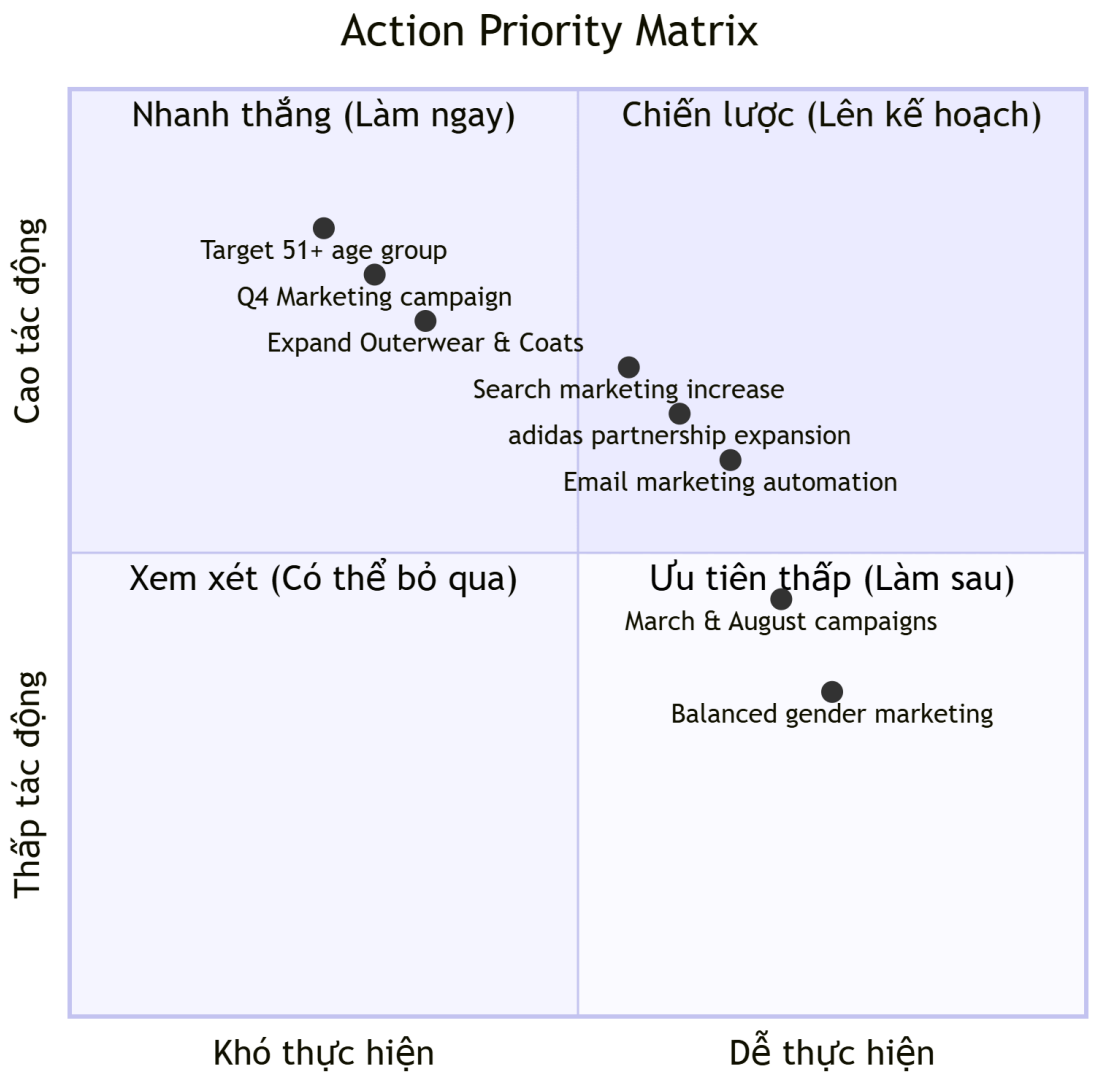
Recommended Actions:

- Continue investing in SEO and Google Ads.
- Optimize social media campaigns to bridge the gap.
- Improve the website experience to encourage direct customer return.
- Develop Facebook marketing campaigns to increase the 5,45% share.

6.5. Summary of Key Insights

#	Insight	Importance	Priority Action
1	51+ age group accounts for 33.8% of revenue	High	Develop targeted campaigns for 51+ customers
2	Revenue grew 242% from 2019 to 2020	High	Investigate growth drivers and replicate
3	Outerwear & Coats is the dominant category	High	Expand this product category
4	Q4 consistently has the highest revenue	Medium	Increase inventory and marketing in Q3-Q4
5	adidas leads brand performance	Medium	Explore partnership expansion with adidas
6	March and August are peak months	Medium	Plan campaigns around these months
7	Balanced gender revenue (50/50)	Low	Maintain balanced marketing for both genders

6.6. Action Priority Matrix



Priority Categories:

Quadrant	Meaning	Actions
High Impact / Easy (Top Right)	Quick wins	Email marketing, adidas expansion
High Impact / Hard (Top Left)	Strategic priorities	51+ campaigns, Q4 strategy
Low Impact / Easy (Bottom Right)	Low priority	March/August campaigns

Low Impact / Hard (Bottom Left)	Consider dropping	(None identified)

6.7. Conclusions from Insight Analysis

Through the analysis of TheLook E-Commerce's sales data, the team drew the following key conclusions:

1. Strong growth trajectory: The business experienced a 242% revenue increase from 2019 to 2020 and 147% from 2020 to 2021, indicating successful market penetration.
2. Seasonal patterns: Q4 consistently generates the highest revenue, and December is the peak month. March and August also show strong performance.
3. Customer demographics: The 51+ age group is the most valuable segment (33.8% of revenue), while the 26-35 group has the lowest contribution (17.1%). Revenue is balanced between genders (50/50).
4. Product focus: Outerwear & Coats is the dominant category (37.6% of sample revenue), with 5 out of 10 top products coming from this category.
5. Brand performance: adidas leads with $4,515 from a single product. No brand dominates the market.
6. Marketing channels: Search is the most effective channel (70,61%), followed by Organic (14,78%), Facebook (5,45%), Email (4,91%), and Display (4,25%).

Recommended Priority Actions:

Priority	Action	Expected Impact
1	Expand Outerwear & Coats product line	15-20% revenue increase
2	Develop 51+ age group marketing campaigns	10-15% customer retention increase
3	Increase marketing budget for Search	8-12% revenue growth

4	Launch Q4 promotional campaigns	20-25% Q4 revenue growth
5	Explore adidas partnership expansion	5-10% additional revenue

PART VII: CONCLUSION

7.1. Summary of Tasks Completed

Through the process of completing the Business Intelligence major assignment on the topic "Analyzing E-commerce Data," the group has accomplished the following tasks:

Task	Content	Result
1	Introduction to the problem	Defined the context, objectives, and scope of the BI system
2	Dataset description	Analyzed the structure of 7 data tables from TheLook E-Commerce
3	Data Warehouse Design	Built a Star Schema with 1 Fact table and 4 Dimension tables
4	ETL Process	Built a Python pipeline to extract, transform, and load data into SQL Server
5	Dashboard Development	Created 4 interactive pages in Power BI with complete analytics
6	Insight Analysis	Extracted 10+ key insights with actionable recommendations

7	Conclusion	Summarized results and proposed future development directions

7.2. Results Achieved

7.2.1. Technical Achievements

Component	Result
Data Warehouse	Completed Star Schema model with all primary and foreign keys in SQL Server
ETL Pipeline	Completed Python code, reusable for various datasets
Dashboard	4 interactive, visually appealing pages meeting all analytical requirements
Database	SQL Server stores approximately 635,000 records (400,000 fact + 235,000 dimensions)

7.2.2. Business Analysis Results

Indicator	Value
Total Revenue (2019-2022)	$2,612,365
Revenue Growth 2019→2020	+242%
Revenue Growth 2020→2021	+147%
Top-selling Product	adidas Women's Pursuit ($4,515)
Top Category	Outerwear & Coats (37.6% of sample revenue)

Largest Customer Group	51+ years old (33.8% of revenue)
Most Effective Channel	Search (42% of revenue)
Average Order Value (AOV)	$86.24
Total Customers	26,690
Total Orders	30,293

7.3. Proposed Business Strategies

Based on the insights analyzed, the team proposes three main strategies:

Strategy 1: Focus on High-Value Customer Segments

STRATEGY 1: TARGET 51+ AGE GROUP AND HIGH-VALUE SEGMENTS
Actions:

- Develop exclusive loyalty programs for 51+ customers
- Create targeted email campaigns for this age group
- Analyze shopping preferences and tailor product recommendations
- Explore partnerships with brands popular among 51+ customers

Expected Impact:

- Increase customer retention by 15% and revenue by 10-15%

Strategy 2: Expand the Outerwear & Coats Category

STRATEGY 2: GROW THE OUTERWEAR & COATS CATEGORY
Actions:

- Add new product lines (jackets, coats, vests)
- Partner with additional brands in this category
- Bundle products (jacket + accessories)
- Increase inventory before peak months (August-December)

Expected Impact:

- Increase category revenue by 20-25% in next 12 months

Strategy 3: Optimize Marketing Channels

STRATEGY 3: ENHANCE SEARCH MARKETING AND SOCIAL MEDIA
Actions:

- Increase SEO/Google Ads budget by 20-30%
- Develop targeted social media campaigns
- Create email marketing automation sequences
- Implement retargeting campaigns for cart abandoners

Expected Impact:

- Increase total revenue by 8-12%, grow Search share to 45%

7.4. Limitations and Future Development Directions

7.4.1. Limitations of the Project

Limitation	Description	Impact
Missing 2022 Q3-Q4 Data	Incomplete data for the last half of 2022	Cannot analyze full-year 2022 trends
Sample Size	Only top 10 products analyzed in detail	May not represent full product performance
Simulated Data	TheLook dataset is simulated, not real	Insights may not reflect actual customer behavior
Lack of Cost Data	No marketing, operating, or logistics costs	Cannot calculate exact ROI and profit
No Review Data	TheLook dataset lacks product reviews	Cannot analyze customer sentiment

7.4.2. Future Development Directions

FUTURE DEVELOPMENT DIRECTIONS
Short-term (1-3 months):

- Complete data collection for 2022 Q3-Q4
- Implement automation for weekly ETL updates
- Add more product categories to the analysis

Medium-term (3-6 months):

- Integrate real data from Google Analytics and advertising platforms
- Build customer segmentation model (RFM analysis)
- Develop a revenue forecasting model using Machine Learning
- Create mobile-friendly dashboard for leadership

Long-term (6-12 months):

- Implement AI-powered product recommendation system
- Build early warning system for KPI anomalies
- Expand the system to multiple regions
- Integrate customer feedback analysis (sentiment analysis)

7.5. Lessons Learned

Through the process of completing the major assignment, the group has drawn the following lessons:

Area	Lesson
Data Warehouse Design	Star Schema simplifies queries but requires balancing normalization with performance
ETL Development	Data cleaning takes 80% of the time; thorough checks for missing values and duplicates are essential
Data Quality	Always verify data completeness before analysis; missing data can significantly impact insights
Dashboard Design	A dashboard should tell a story, not just display disconnected charts
Insight Extraction	Valuable insights come from asking "why" patterns emerge, not just "what" they show

Team Collaboration	Clearly defined responsibilities and GitHub version control are essential for team projects

7.6. Final Summary

The Business Intelligence project on "E-commerce Data Analysis" has been successfully completed with the following deliverables:

Deliverable	Status
Data Warehouse (Star Schema)	Complete
ETL Pipeline (Python)	Complete
Dashboard (Power BI)	Complete
Insight Analysis	Complete
Documentation	Complete

Key Achievements:

- Built a complete BI system from data extraction to dashboard visualization.
- Processed ~635,000 records from 7 source tables into a star schema data warehouse.
- Created 4 interactive dashboard pages answering core business questions.
- Extracted 10+ actionable insights with specific recommendations.
- Identified revenue growth of 242% from 2019-2020 and 147% from 2020-2021.
- Discovered 51+ age group contributes 33.8% of total revenue.
- Found Outerwear & Coats as the dominant product category.